

Semester Thesis

Enhancement of bipedal locomotion using Alma's manipulator as a support leg

Spring Term 2023

Intellectual Property Agreement

The Student is acting under the supervision of Prof. Hutter and contributing to research and development in the Research Area at ETH Zürich. Research results of students outside the scope of an employment contract with ETH Zurich belong to the student him- or herself. Results of his collaborations ("Research Results") may be exploited by ETH Zurich, possibly together with other results that are created at ETH Zurich. To facilitate and to enable a common exploitation of all combined research results, the Collaborator hereby assigns his rights to his Research Results to ETH Zurich. In exchange, the Collaborator shall be treated like an employee of ETH Zurich with respect to any income generated due to his Research Results.

This Agreement regulates the participation of the Collaborator in research and development within the above mentioned Research Area and the rights to the Research Results being created.

1. Collaboration within the Research Area

- (1) The Student agrees to keep all research results confidential as well as other information from the research group of Prof. Hutter that is declared as confidential. This obligation to confidentiality shall persist until he or she is informed by ETH Zurich that the intellectual property rights to the research results have been protected through patent applications or other adequate measures or that no protection is sought. However, this obligation will not extend over a period exceeding 12 months after completion of his collaboration with Prof. Hutter.
- (2) The Collaborator will make available all information, data and research results out of the collaboration to ETH Zurich.

2. Intellectual Property Rights

- (1) The Student assigns his rights to the Research Results, including inventions and works protected by copyright, but not including his moral rights ("Urheberpersönlichkeitsrechte"), to ETH Zurich. Herewith, he/she cedes, in particular, all rights for commercial exploitations of Research Results to ETH Zurich. He/She is doing this voluntarily and with full awareness and he/she acknowledges that there are other options to participate in projects at ETH Zurich that do not require such assignment of rights.
- (2) In exchange, he will be compensated by ETH Zurich in the case of income through the commercial exploitation of Research Results. Compensation will be made as if he was an employee of ETH Zurich and according to the guidelines "Richtlinien für die wirtschaftliche Verwertung von Forschungsergebnissen der ETH Zürich".
- (3) If a patent application is filed for an invention based on the Research Results, the Collaborator will duly provide all necessary signatures. He also agrees to be available whenever his aid is necessary in the course of the patent application process, e.g. to respond to questions of patent examiners or the like.

3. Amendments and Additions

Amendments and/or additions of this Agreement can be made in writing, only.

4. Settlement of Disagreements

Should disagreements arise out between the parties, the parties will make an effort to settle them between them in good faith.

In case of failure of these agreements, Swiss Law shall be applied and the Courts of Zurich shall have exclusive jurisdiction.

The Collaborator

Zurich, 08/03/2023

YASSIER BELGUERCHI
Name

Signature



Declaration of Originality

I hereby declare that the written work I have submitted entitled
Enhancement of bipedal locomotion using Alma's manipulator as a support leg is original work which I alone have authored and which is written in my own words.

Author(s): Belguerch Yassir

Supervisor(s): Fan Shi, Yuntao Ma

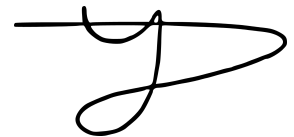
Programme tutor: Roland Siegwart

Supervising lecturer: Marco Hutter

With the signature, I declare that I have been informed regarding normal academic citation rules and that I have read and understood the information on 'Citation etiquette' (<https://www.ethz.ch/content/dam/ethz/main/education/rechtliches-abschluesse/leistungskontrollen/plagiarism-citationetiquette.pdf>). The citation conventions usual to the discipline in question here have been respected. I declare that I have documented all methods, data, and processes truthfully. I have mentioned all persons who were significant facilitators of the work.

The above-written work may be tested electronically for plagiarism.

Signature:

A handwritten signature in black ink, consisting of a horizontal line followed by a stylized, looped flourish.

Abstract

Our semester project focuses on enhancing the bipedal locomotion capabilities of the Alma robot, a quadrupedal robot with a manipulator. By using the manipulator as a support leg, we aim to free the front legs for manipulation tasks, enabling more complex actions like object transportation in warehouses. The project is divided into linear and angular velocity tracking sub-tasks to achieve precise movement to a desired point in the xy plane.. While we rely on simulation data for relative position and yaw angle information, real-life applications would involve implementing additional perception methods. Our goal is to explore the potential of bipedal locomotion for more complex manipulation tasks.

Acknowledgements

I would like to thank my supervisors Fan Shi and Yuntao Ma for their advices, patience and guidance throughout the project. I would also like to thank Prof. Marco Hutter for giving me the possibility to do this project in his lab.

Contents

1	Introduction	8
2	Problem Formulation	9
3	Method	10
3.1	Markov Decision Process	10
3.2	Actor-Critic Methods	10
3.3	Reward Functions	11
3.4	Multi-task Framework	13
3.4.1	Progressive Learning	14
3.4.2	Implementation	15
3.5	Dual Policy Locomotion	17
4	Experimental Setup	19
4.1	Starting Configuration	19
4.2	Addition of a wheel to the end-effector	19
4.3	Training environment	20
4.3.1	Starting configuration	20
4.4	Joints used for locomotion	21
4.5	Observation Space	21
4.6	Package grabbing environment	22
5	Results and Analysis	23
5.1	Linear Velocity Tracking	23
5.2	Angular Velocity Tracking	24
5.3	Multi-task Framework	24
5.4	Comparison between the two methods	25
5.5	Payload	26
6	Discussion	29
6.1	Upside Down Configuration Comparison	29
6.2	Torque in knee during translation	30
7	Conclusion	30
8	Appendix	31

Contents

1 Introduction

ANYmal is known to be a quadrupedal robot with excellent quadrupedal locomotion that enables it to move in all directions and climb stairs consistently and safely. This feature makes it an efficient robot for performing autonomous inspection and exploration tasks. On the other hand, the Alma robot utilizes a manipulator to grasp objects and carry out manipulation tasks.

In our semester project, we delved into the potential of utilizing the manipulator to enable the robot to adopt a bipedal configuration. This configuration opens up the possibility of utilizing the two front legs for manipulation purposes, including object transportation, which could be useful in environments such as warehouses.

Bipedal locomotion of the ANYmal robot has already been studied using the addition of wheels on the the feet end effectors (fig 3), which turned out to be a good solution for hybrid bipedal and quadrupedal locomotion. However, the robot needs space to balance itself and is not suited to logistic tasks such as carrying a box since it is not statically stable.



Figure 1: ANYmal robot equipped with wheels

Reinforcement learning has demonstrated its effectiveness in acquiring control policies for quadrupedal locomotion, as evidenced by the notable achievements in [ref1]. In their work, the authors trained a neural network to grasp the actuator dynamics of the ANYdrive actuators employed in the ANYmal robot. By incorporating this actuator network into the reinforcement learning pipeline during simulation, they successfully mitigated the sim-to-real gap and even achieved one-shot policy deployment. It can indeed be considered as the most robust control technique, with a very efficient framework.

In our semester project, we adopted a similar reinforcement learning pipeline with certain modifications to facilitate bipedal locomotion, as we will delve into later.

2 Problem Formulation

The objective of this project is to investigate potential methods for enhancing the bipedal locomotion of the Alma robot to facilitate more intricate manipulation tasks. The core concept of this project involves leveraging the Alma robot’s manipulator as a means of support during bipedal locomotion. By utilizing the manipulator for stability, the two front legs of the robot can be freed to perform tasks like manipulation or transport, thereby enabling more complex and versatile actions.

The ultimate goal of this project is to achieve precise movement to a desired point in the xy plane utilizing the enhanced bipedal locomotion capabilities. However, training the robot directly to reach the target point from scratch using a distance-based reward function is a challenging task. To address this, the project is divided into two separate tasks: linear velocity tracking and angular velocity tracking. By breaking down the problem into these sub-tasks, the robot can learn and refine its locomotion skills more effectively, leading to improved performance in reaching the desired point.

One significant challenge we may encounter is the increased torque requirement for the hip and knee joints when using only two feet for locomotion, as opposed to the traditional four-legged locomotion. The actuators in the hip and knee joints will need to apply much higher torque, which could exceed the torque limits of these actuators. However, we can mitigate this issue by utilizing a manipulator as a support. Although the manipulator’s actuators have a lower torque limit than the leg actuators, they can provide additional stability and support. Furthermore, it is important to note that tripodal locomotion, unlike the more commonly observed quadrupedal locomotion in nature, is relatively rare. As a result, ensuring the realism and feasibility of the locomotion we achieve becomes crucial for practical implementation in real-life scenarios. We must carefully design and fine-tune the locomotion gaits to ensure they align with the constraints and capabilities of the system.

In this semester project, we are not focusing on exploring or studying perception aspects. Instead, we rely on simulation data that provides us with information about the relative position and yaw angle of the robot with respect to the goal point. This simulated information serves as a substitute for real-time perception in our project. It’s important to note that in a real-life setup, achieving such relative position and orientation information would typically involve employing perception methods like camera calibration or other relevant techniques. These methods would enable the robot to gather data from its environment and make informed decisions based on real-time perception.

3 Method

To tackle RL problems, a mathematical framework called Markov Decision Processes (MDPs) is often employed. MDPs provide a formal representation of the environment, allowing us to model the interactions between an agent and its surroundings. In the first two subsections, we introduce the concepts of MDPs and explore the Actor-Critic methods used in our reinforcement learning pipeline, which combine the benefits of both value-based and policy-based approaches. Later we describe the methods we used for our project.

3.1 Markov Decision Process

MDPs serve as a foundation for RL, enabling us to describe and analyze decision-making problems. Key components of an MDP include the state space, action space, transition probabilities, reward function, and discount factor. The state space describes all possible states that the agent can encounter. The action space represents the set of actions available to the agent at each state. Transition probabilities specify the likelihood of transitioning from one state to another based on the chosen action. The reward function provides immediate feedback to the agent, guiding its behavior, while the discount factor determines the importance of future rewards. In a Markov Decision Process (MDP), the Markov property states that the probability of being in a particular state depends only on the previous state, disregarding the entire history of states leading up to it. Specifically, the probability of transitioning to state s' at time t , given the previous state s and the action taken a , is determined solely by the transition function $T(s, a, s')$:

$$P(S_t = s' | S_{t-1} = s, a_t = a) = T(s, a, s')$$

Each MDP also incorporates a reward function $R : S \mapsto R$ that assigns a numeric value to each state $s \in S$. The objective of a Markov Decision Process is to find a policy, which is a mapping from states to actions. The policy function $\Pi : S \mapsto A$ selects the appropriate action a from the $a \in A$ based on the current state s . The goal of the policy is to maximize both the immediate reward $R(s)$ and the cumulative value of future rewards.

Therefore, the main objective in an MDP is to determine an optimal policy that maximizes the expected total reward over time, taking into account the probabilistic nature of state transitions and the rewards associated with different states.

In our reinforcement learning pipeline, we use the PPO algorithm. It maintains stability by introducing a clipping mechanism to control policy updates. Efficiently exploring the action space, PPO converges to robust policies, enabling agile locomotion patterns and effective navigation in diverse terrains. Its simplicity, scalability, and suitability for continuous action spaces make PPO a favored choice for training our tasks. The hyperparameters used in this algorithm are listed in [15](#)

3.2 Actor-Critic Methods

Actor-Critic methods are widely used in RL due to their ability to combine value-based and policy-based approaches. These methods consist of two essential components: the

actor and the critic. The actor is responsible for selecting actions based on a policy, which defines the agent’s decision-making behavior. On the other hand, the critic evaluates the value function and provides feedback to the actor, helping to refine its decision-making process.

3.3 Reward Functions

In the reinforcement learning pipeline used to train our robot, the reward computation plays a crucial role. The table below provides an overview of the reward functions utilized in our reward configuration for different tasks. Each of these functions should return a torch tensor with a size equal to the number of environments we are training. To simplify the descriptions, we will demonstrate the computation of rewards for a single environment.

Table 1: Function Descriptions

Function Name	Task_code / Role
Tracking_ang_vel_standing	1 / Tracking the angular velocity (yaw rate)
Dist_from_goal	3 / Rewards the robot when it is closer to the goal point
Tracking_lin_vel_standing	2 / Tracking the velocity of the base in the x direction
Wheel_contact_force	0 / Rewards when wheel in contact with the ground
Feet_air_time	0 / Rewards feet air time
Feet_velocities	0 / Penalizes linear velocities of feet
Knees_velocity	0 / Penalizes knees joint velocity
Foot_slippage	0 / Penalizes foot slippage
Base_height	0 / Gives penalty when the base height isn’t right
Flat_orientation	0 / Penalizes flat orientation
Termination	0 / Penalizes termination of the episode
bad_orientation	0 / Penalizes bad pitch angle orientation
Link_linear_velocity	0 / Penalizes linear velocity of desired links
Torques	0 / Penalizes torques
Dof_acc	0 / Penalizes DOF acceleration
Collision	0 / Penalizes Collision
Stand_still	0 / Penalises difference from starting configuration

Tracking_ang_vel_standing : The goal of this function is to allow the robot to track an angular velocity command, to be able to perform a rotation in the xy plane. In previous works, the yaw angle was around the z axis in the base frame. However in our bipedal configuration the rotation should be done around the x axis in the base frame. Since we are training multiple asks such as linear velocity tracking at the same time, we also need to penalise undesired motions such as velocities in x or y directions.

The value is calculated following:

$$Reward = c_1 * e^{-(yaw_{command} - yaw_{velocity})} + c_2 * e^{-x_{velocity}} + c_2 * e^{-y_{velocity}}$$

where:

$$c_1 = 1, c_2 = 0.2$$

Tracking_lin_vel_standing : The goal of this function is to allow the robot to track a linear velocity command, to be able to perform a translation in the x direction. In previous works, the x direction was the same in the base frame whereas in our case this direction is in the previous z direction. Since we are training multiple asks such as angular velocity tracking at the same time, we also need to penalise undesired motions such as angular velocities, the yaw rate.

$$Reward = c_1 * e^{-(x_{command} - x_{velocity})} + c_2 * e^{-yaw_{rate}} + c_2 * e^{-y_{velocity}}$$

where:

$$c_1 = 1, c_2 = 0.2$$

Dist_from_goal : This function gives a higher reward when the robot gets closer to the goal position, the reward computation is hence very simple.

$$Reward = c_1 * e^{-(x_{goal} - x_{robot})^2} + c_2 * e^{-(y_{goal} - y_{robot})^2}$$

where:

$$c_1 = 1, c_2 = 1$$

Wheel_contact_force : This function gives a boolean reward (returns 1 when the conditions are met). The goal of this function is to reward the usage of the wheel during locomotion, which will be useful when carrying weight (one more contact point with the ground).

$$Reward = |wheel_{force}| > 0.1$$

Where $wheel_{force}$ is the force applied on the wheel in the z direction.

Feet_velocities : This function is used to penalize high feet velocities when performing locomotion to encourage realistic motions.

$$Reward = \sum_{i=1}^2 ||hind_feet_{vel}||_2$$

Knees_velocities : This function is used to penalize high feet velocities when performing locomotion to encourage realistic motions.

$$Reward = \sum_{i=1}^2 ||hind_knee_{vel}||_2$$

Base_height : This important function penalizes the difference between the height of the base and desired height. Before the addition of this function, the robot wouldn't learn how to stand. After setting a desired height of approximately 0.75 *cm*, the robot is able to learn the standing position.

$$Reward = (base_{height} - target_{height})^2$$

Flat_orientation : This function is used to give reward when the robot orientation is in a standing configuration, zero penalty is given when the base is perpendicular to the ground plane. Since we directly have access to the projected gravity on the base, we can simply use the norm of the projected gravity in the base frame.

$$Reward = ||projected_{gravity}||_2^2$$

Link_linear_velocity : This function is used to penalize velocity of a given link given as parameter.

$$Reward = ||velocity_{link}||_2^2$$

Stand_still : This function is used to penalize the difference between the initial desired position of the robot and the current one during the simulation.

$$Reward = ||dof_pos_{desired} - dof_pos_{env}||_2^2$$

Termination : This function is used to terminate the on going episode if the robot finds itself in an undesired position. The return of this function is a boolean vector of size num_envs and terminates the episode when the value is true. In a bipedal configuration we determined that the robot should terminate whenever one of the bodies that is not the hind feet enters in contact to the ground or if there is an internal collision of the robot. We have access to the the contact forces of all bodies so we could simply check if these contact forces are not zero. However the contact detection is not precise when it comes to the edges of the 3D collision bodies, which caused the robot to learn undesired policies. The solution is to couple the contact detection to a height control, so that the robot also terminates when the height of the selected body is lower than a certain threshold.

$$Termination = (||F_{bodies}|| > F_{ct}) \text{ or } (||H_{bodies}|| < H_t)$$

Where F_{bodies} are the forces applied on the following bodies : base, thighs for the hind feet, and arms of the manipulator, and H_{bodies} the heights of the following bodies : knees, hind and front hips.

Bad_Orientation : This function is used to terminate the on going episode if the robot finds itself in an undesired orientation. The return of this function is a boolean vector of size num_envs and terminates the episode when the value is true. in our case we want the orientation to around y and x to be around $\pi/2$. We therefore have $\theta_{inf} = \pi/2 - \epsilon$ and $\theta_{sup} = \pi/2 + \epsilon$ with $\epsilon = 0.7rad$

$$Bad_orientation = (\theta_x < \theta_{inf}) \text{ or } (\theta > \theta_{sup}) \text{ or } (\theta_y < \theta_{inf}) \text{ or } (\theta > \theta_{sup})$$

3.4 Multi-task Framework

Once we got satisfying motions for both linear and angular velocity, we explored a way to combine both of these motions using a single policy. The idea that we got is that the robot should be able to directly go to certain coordinates in the xy plane using the two motions that have been learned before. However, after training the robot on only reaching the desired coordinates, the results weren't satisfying at all because training directly a robot on such task is too hard for him to learn. This is why we decided to

implement **progressive learning** to the already existing framework detailed in [1]. This progressive learning pipeline is inspired from works [4] and [16] and is implemented in a simpler way and in a continuous reward setup.

3.4.1 Progressive Learning

The competence of task k can be quantified by measuring its performance or success rate. One way to evaluate the competence of a task is by using a reward-based system. The reward can be defined as a scalar value that reflects the level of success achieved in completing the task.

For example, in a reinforcement learning setting, the agent can receive positive rewards for successfully completing the task and negative rewards for failures or sub-optimal performance. The cumulative reward over a certain period can serve as a measure of the agent’s competence in that task.

To determine the competence of task k , we can consider the average reward obtained by agents or individuals that have been trained on that task. The higher the average reward, the more competent the task is considered to be.

By tracking the competence of each task, we can design a probability distribution that governs the sampling of tasks for training. Initially, the distribution can be biased towards simpler tasks to allow agents to learn foundational skills. As the performance on these simpler tasks improves, the distribution can be gradually shifted towards more challenging tasks, utilizing the knowledge and skills acquired during the earlier stages. The probability distribution for task sampling can evolve over time based on the rewards and training duration.

This evolution can be designed to favor tasks that have higher rewards and allocate more training time to them. By dynamically adjusting the probability distribution, the training process can focus on tasks that provide the most learning progress and maximize the overall efficiency of training.

$$C_k = \frac{\sum_{i=1}^{n_r} rew_{ki}}{\sum_{j=1}^{nb_{tasks}} \sum_{i=1}^{n_r} rew_{ji}} \quad (1)$$

Where n_r is the number of reward functions used in the given task and rew_{ki} the reward given by the i^{th} reward function for task k . We normalize to have a number between 0 and 1.

To quantify the learning speed or progress rate of a task, we can calculate the derivative of its competence over time. This derivative provides information about how quickly the task’s competence is changing. A positive derivative indicates fast progress, while a negative derivative suggests slower improvement or decline in competence, which shouldn’t happen in our case since if the reward functions are well designed we should converge towards a higher or stagnating reward. Monitoring the derivative allows us to identify tasks that are being learned quickly or those that require additional attention. The learning progress of task k at iteration t is given by :

$$LP_{kt} = \frac{\partial C_k}{\partial t} = C_{kt} - C_{k(t-1)} \quad (2)$$

However, since the rewards can have very abrupt changes because we are using all the population, we proceed to average the derivatives by using w_{avg} previous competence

values, this creates a slower change in the probability distribution but doesn't affect the results. The learning progress of task k at iteration t is computed using the following formula :

$$LP_{kt} = \frac{1}{w_{avg}/2} \left(\sum_{\tau=t-w_{avg}/2}^t C_{k\tau} - \sum_{\tau=t-w_{avg}}^{t-w_{avg}/2} C_{k\tau} \right) \quad (3)$$

Finally, the probability of sampling a specific task k , can be determined by dividing its learning progress by the sum of the learning progresses of all tasks:

$$P_k = \frac{LP_k}{\sum_{j=1}^{nb_{tasks}} LP_j} \quad (4)$$

However, in early stages of training, initial rewards are low, not stable, and not yet significant. Adding a random component helps maintain the exploration of all tasks in the early stages and also in later stages of the training. Consequently, the revised equation for task sampling probability can be written as:

$$P_k = \epsilon * \left(\frac{1}{nb_{tasks}} \right) + (1 - \epsilon) \frac{LP_k}{\sum_{j=1}^{nb_{tasks}} LP_j} \quad (5)$$

The first term is simply as random sampling probability given by the number of tasks. The term ϵ is the weight of two components, its value is typically 0.3 (found empirically). It can easily be shown that :

$$\sum_{j=1}^{nb_{tasks}} P_j = 1 \quad (6)$$

3.4.2 Implementation

In this section, it would be way too long to explain all the changes and new code that has been written, so we chose to only highlight the most relevant changes and additions to the already existing pipeline, in a simplified way.

First, we describe a task as set of reward functions that are listed in 3.3. Each task has a main reward function, in our case the three main reward functions are given by the tracking of linear velocity, tracking of angular velocity, and the distance from the goal. The other rewards functions are used to shape the bipedal locomotion and to perform realistic motions, hence they are all part of the three tasks. To identify to which task a reward function belongs, we create a parameter given to each function which is the *task_code*. Table 3.3 describes the *task_code* for each function. For optimization, code 0 is given to redundant reward functions that are used in every task and a positive integer to the principal task reward functions.

The entire previous pipeline was designed to train all the agents to the same reward functions, which are part of a unique task. This is why all the reward functions returns are a vector that has the size of the number of environments, which is optimal for total reward computation in that case. However in our case the reward computation of a single environment is task dependent, which means we can't simply sum all the rewards. We could iterate over all the existing environments (4096 in our experimental

setup) and sum only the relevant functions, but this is too much computational time consuming since we are adding a for loop. In fact, the computation time becomes so high that it is of the same order of the refresh rate of the simulation. A solution is to create a mask that we multiply by the computed reward that has a size of num_envs to select the relevant functions.

For example, if we have 2 environments and are computing the reward of two functions that are only part of the task the second environment is training on, the $task_mask$ will be of size $(num_envs \times num_tasks) : (2 \times 2)$ and equal to $\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$, when computing the rewards of the said tasks, we multiply the return of the reward function by the corresponding columns.

The following table describes how this method is used to compute the reward in our multi-task framework where the reward is the total reward computed and episode_sums a dictionary that helps keep track on the rewards of the individual functions :

Code	Pseudo Code
1	Initialize reward variables <code>reward = array(num_envs, num_tasks)</code> <code>#dictionary responsible for keeping track of the rewards</code> <code>#for each function in each task</code> <code>episode_sums = dictionary {...</code> <code>task : [function for function in task]}</code> <code>for each task in task_dictionary:</code> <code> reward[:, task] = zeros(num_envs, 1)</code>
2	Calculate rewards for each task <code>for each task in task_dictionary:</code> <code> for each name, function in self.task_dict[task].items():</code> <code> if name is "termination":</code> <code> continue # Skip to the next iteration</code> <code> rew = compute_rew(env) * rew_scale * task_mask[:, task]</code> <code> reward[:, task] += rew</code> <code> episode_sums[task][name] += rew</code> <code># Add termination reward after clipping</code> <code>if "termination" in self.task_dict[task]:</code> <code> rew = compute_rew_termination(env) * ...</code> <code> rew_scale * task_mask[:, task]</code> <code> reward[:, task] += rew</code> <code> episode_sums[task]["termination"] += rew</code> <code>return reward</code>

Another important addition are the new observations states for our framework. The already existing observation states are listed in 4.5. In our case we need new observations for this multi-task training. The robot needs to know which task it is training on, which why we add a task encoder to the observations, it also needs to know its relative position to the goal and the desired yaw angle we want the agent to have when reaching the goal point. The new set of observations becomes :

- Base linear velocity expressed in floating base coordinates : (num_envs x 3)
- Base angular velocity expressed in floating base coordinates : (num_envs x 3)
- Gravity vector expressed in floating base coordinates : (num_envs x 3)
- Joint positions : (num_envs x 16)
- Joint velocities : (num_envs x 16)
- Velocity commands
- **Task encoder, indicating the current task being trained (num_envs x 1)**
- **x goal coordinate in the base frame (num_envs x 1)**
- **y goal coordinate in the base frame (num_envs x 1)**
- **Relative yaw angle to the desired one (num_envs x 1)**

3.5 Dual Policy Locomotion

As shown in section , the results of Multi-task framework were not satisfying, we hence decided to use two policies instead of one to control our robot. In this framework, we train two different policies that correspond to the two tasks of linear and angular velocity tracking of the multi-task framework.

In fact, another way to combine both linear and angular velocity to reach a desire point in the xy plane is to use a simple controller that controls when to use the first or the second policy. There are multiple ways to design this controller, we went for the easier one since the purpose of this project is to show that this locomotion is possible. The desired locomotion is to have the robot perform rotation around the yaw angle and then move straight to the desired point using linear velocity tracking in the x direction in the base frame. As explained in the problem formulation, we assume we have access to the yaw angle and relative position to the goal. We intuitively want the robot to rotate towards the goal, the desired yaw angle θ^* is simply given by :

$$\theta^* = \tan^{-1}\left(\frac{y_{goal} - y_{robot}}{x_{goal} - x_{robot}}\right) \quad (7)$$

We then use a simple PD controller where the yaw rate of the robot using the angular velocity tracking policy previously trained at time t is given by :

$$\dot{\theta}_t = k_P * (\theta_t - \theta^*) + k_D * (\theta_t - \theta_{t-1}) \quad (8)$$

Once the difference between the yaw angle of the agent and the desired yaw angle is lower than a certain threshold, we switch to the linear velocity tracking policy. The problem is also one dimensional in this case because the robot only moves linearly in the x direction in the base frame. We can also use a simple PD controller that uses the distance to the goal :

$$\dot{x}_t^b = k_P * (x_t^b - x_{goal}^b) + k_D * (x_t - x_{t-1}) \quad (9)$$

Where x_t^b is the distance between the goal and the robot. One thing to note is that since the robot is already position towards the goal (yaw angle) the distance is always positive and given by :

$$x_t^b = \sqrt{(y_{goal} - y_{robot})^2 + (x_{goal} - x_{robot})^2} \quad (10)$$

4 Experimental Setup

This section outlines the experimental setup, including the environment, sensors, actuators, and any modifications made to the robotics platform.

4.1 Starting Configuration

For our semester project, we didn't explore or attempt to solve the problem of making the robot stand up by itself. As a result, the robot spawns directly in a standing configuration that is quite different from the quadrupedal locomotion starting configuration. To address this, we took inspiration from how a kangaroo uses its tail to move forward. In our case, we used the Alma manipulator as a tail and positioned the robot so that its two hind feet are in contact with the ground. Researchers in University of Colorado studied the way Kangaroos use their tails to move in a linear direction [5]. They use their tails as a support to push upward and jump forward with their two hind legs (video can be found in 8).

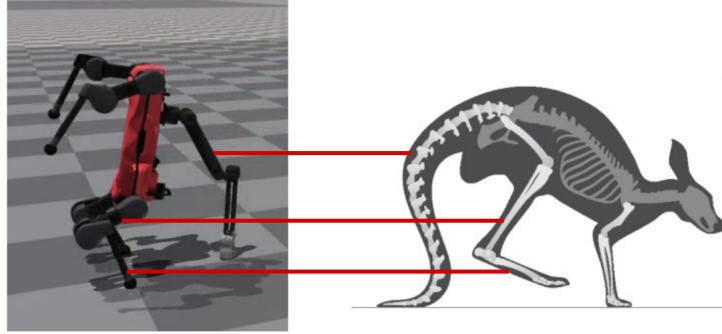


Figure 2: Comparison between kangaroo and starting position. Image : Heather More, Simon Fraser University

We made the decision to set the knee joint at a 90° angle, despite the higher torque it causes, because the weight is primarily applied to the hip rather than the leg due to the length of the manipulator. If the angle of the knee joint were higher, such as 110° , the manipulator would have to be lifted from the ground. However, we don't want this since we require three contact points with the ground for stability.

4.2 Addition of a wheel to the end-effector

In our configuration, the end-effector of the manipulator remains in contact with the ground throughout the rotation and translation movements of the robot. However, we encountered unwanted behavior during simulations due to friction between the end-effector and the ground. This friction caused the robot to fall or hindered its rotation.

To mitigate this issue, we devised a solution by adding a caster wheel directly connected to the actuated forearm of the manipulator. The caster wheel is mounted

on the forearm’s revolute joint in the z direction. This addition serves two purposes. Firstly, during the rotation movement, the caster wheel traces a circular path around the robot as the base turns for yaw angle rotation. This aids in the rotational motion by reducing the friction and allowing smoother rotation. Secondly, the caster wheel helps stabilize the translation in the x direction. When the two hind feet of the robot are translating, the caster wheel supports the base and prevents it from falling. This additional contact point enhances stability during translation movements, ensuring that the robot remains balanced and does not tip over.

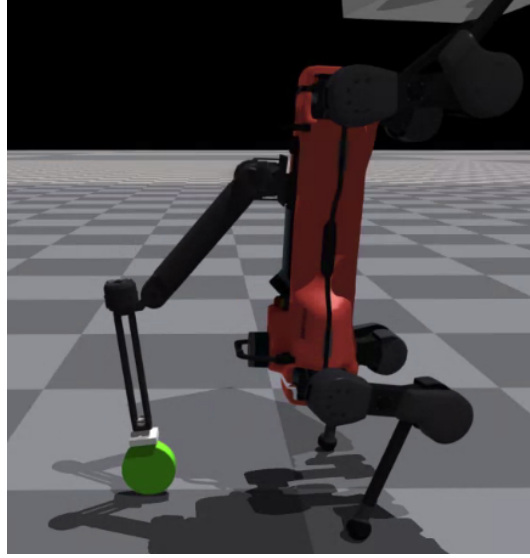


Figure 3: Robot with added wheel, dimensions: cylinder of radius 0.08 cm and length 0.05 cm

By incorporating the caster wheel into the manipulator design, we aim to address the unwanted behavior caused by friction and improve both the rotational and translational capabilities of the robot. The dimensions and friction coefficients of the wheel were inspired by the already existing ANYmal on wheels.

4.3 Training environment

4.3.1 Starting configuration

We chose to train 4096 robots at a time during the training for our locomotion tasks. The robots are separated by 3m in x and y direction. The starting configuration is that of a robot already standing with its manipulator directed towards the ground, its front feet in a random position (not relevant) and hind feet in contact with the ground. The robot is spawned 5 cm higher than the ground (distance between feet and ground) to avoid unrealistic collisions with the ground of the wheel (wheel spawning inside the ground).

4.4 Joints used for locomotion

In this section we are looking into which actuators are used to perform locomotion. Since we decided to go with two different types of locomotion, each task might not use the exact same actuators as the other. The following figures show the actuators used for the two types of locomotion : translation in x direction and rotation around z axis (yaw angle).

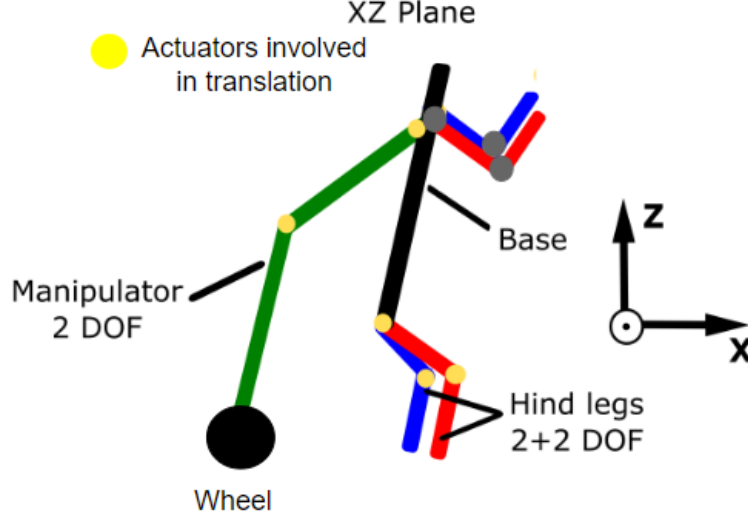


Figure 4: Actuators involved in x direction translation

In the system described in 4, the actuators responsible for translation in the x -direction are the knee and hip actuators, which operate around the y -axis. Additionally, two actuators from the manipulator are employed to lift and position the wheel correctly. Therefore, the total number of actuators used for translation is 2 (left foot) + 2 (right foot) + 2 (manipulator) = 6 .

Regarding rotation in 5, the actuators involved are the knee joints, which are also utilized for translation, the hip actuators revolving around the z -axis, and the actuator directly connected to the wheel for guidance. Consequently, the total number of actuators for rotation is 2 (left foot) + 2 (right foot) + 1 (manipulator) = 5 .

For the overall locomotion, we have 6 actuators for translation and 5 actuators for rotation. However, we subtract 2 redundant actuators that are used for both translation and rotation, resulting in a total of 9 actuators.

As for manipulation, the two front legs provide 6 actuators that can be utilized for manipulation tasks.

4.5 Observation Space

The list below provides the definition of the observations that are given to the reinforcement learning network and their corresponding dimensions. The value *num_envs* indicates the number of environment we are using to train our policy.

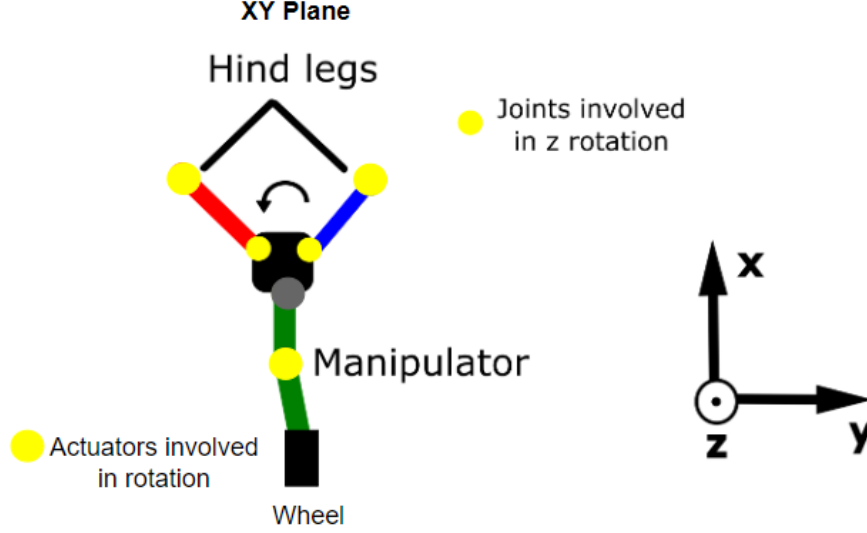


Figure 5: Actuators involved in rotation around z axis

- Base linear velocity expressed in floating base coordinates : $(\text{num_envs} \times 3)$
- Base angular velocity expressed in floating base coordinates : $(\text{num_envs} \times 3)$
- Gravity vector expressed in floating base coordinates : $(\text{num_envs} \times 3)$
- Joint positions : $(\text{num_envs} \times 16)$
- Joint velocities : $(\text{num_envs} \times 16)$
- Velocity commands

4.6 Package grabbing environment

To measure the robot’s ability to carry a specific payload in our configuration, we developed a new training environment distinct from the one used for locomotion training. In this environment, the robot is trained to carry a box-shaped package that appears above it. To achieve this, we employed two reward functions during training. The first reward function focuses on maintaining a suitable base height, which is similar to the one utilized in the original training (mentioned in section 3.3). This ensures that the robot maintains stability and prevents it from falling. The second reward function penalizes undesired collisions that may occur during the episode, such as knees or hands touching the ground, or if the height of the package drops, indicating that the robot has dropped it. This serves as a termination condition for the ongoing episode.

To assess the robot’s ability to carry different payloads, we manipulated the density of the package. By varying the density, we can effectively measure the payload capacity of the robot. We follow a progressive training approach where we first train a policy using a relatively high density for the package. Once this policy has been established, we can resume training from this point when we decrease the density of the package.

This method of gradually decreasing the density of the package significantly reduces the training time required for this task. By building upon the knowledge and skills acquired in the previous training phase, the robot can adapt more quickly to the changes in payload density, allowing for a more efficient training process.

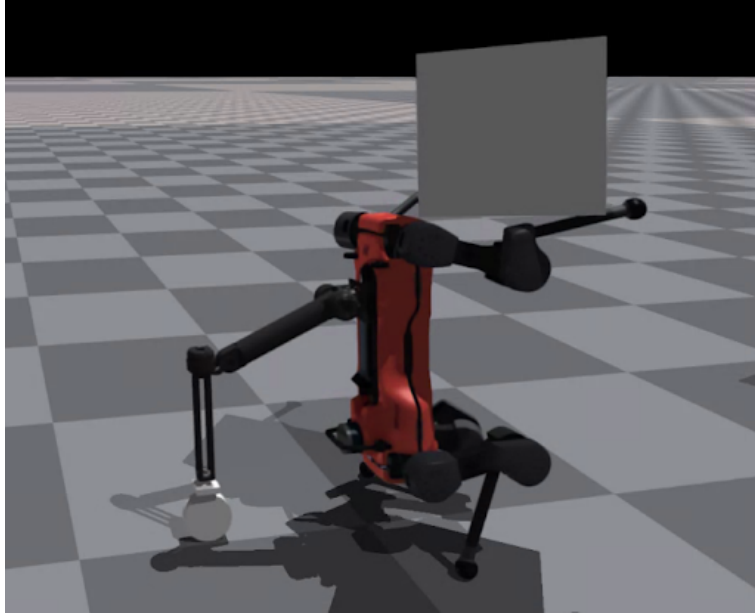


Figure 6: Robot training on package grabbing task

5 Results and Analysis

5.1 Linear Velocity Tracking

Figure 7 describes how well our robot tracks linear velocity, in orange is the robot base velocity in the x direction and in blue is the command given to the robot. As we can see in the the figure the mean of the robot’s base velocity follows the command, however there is a high variations with spikes for the three different speeds tested on. This consequent spikes are due to the fact that the robot is constantly jumping which creates this discontinuity in the linear velocity. This is therefore not an undesired trait since this locomotion is inspired by that of Kangaroo as explained in 4.1. One important point to note is that in the case of the Kangaroo, the force applied to the tail is much greater than the force applied to the hind legs. However, in our scenario, the actuators used on the manipulator have a lower torque limit compared to the ones in the robot’s hind legs. As a result, when we attempted to replicate the kangaroo’s movement, the robot exhibited unrealistic motion.

To address this issue, we devised a simple solution: we made the robot move backwards instead of forward. By doing so, the robot utilizes its hind legs to jump and push backward, allowing for a realistic motion without exceeding the torque limits of the manipulator’s actuators. This approach ensures that the robot operates within its specified torque capabilities while still achieving a lifelike motion resembling that of a

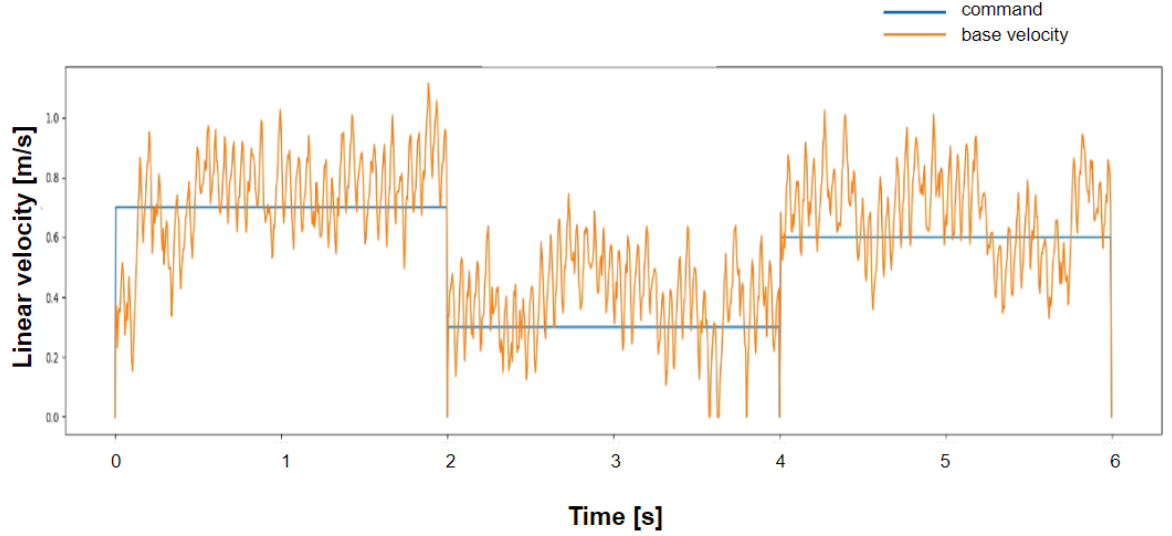


Figure 7: Linear velocity tracking plot

kangaroo but in reversed way. Video of the robot during the tracking test can be found in the appendix.

Results showed that the robot would produce unrealistic motion when the robot moves at a speed that is lower than a certain threshold : 0.25 m/s, and this is due to the fact that when the jumping of the robot induces a minimal speed, which is a similar aspect to gait changes with quadrupedal animals. When the commanded speed isn't higher than the threshold, the robot moves linearly by lifting one foot at a time instead of both of them, which causes the a much higher speed on the feet to be able to keep the robot stable.

5.2 Angular Velocity Tracking

Figure 8 shows how well the robot is capable of tracking angular velocity, in orange is the robot base angular velocity around the z axis and in blue is the command given to the robot. We can see that the tracking is quite precise compared to the linear velocity tracking , and that the robot is capable of changes with a relatively high frequency. The video of angular tracking can be found in the section 8. The video also shows the robot is making use of the revolute joint in the elbow of the manipulator to direct the wheel to turn around in one or the other direction.

5.3 Multi-task Framework

The objective of progressive learning was to train three locomotion tasks (angular velocity tracking, linear velocity tracking, and reaching a goal in the xy plane) using a single policy. To achieve this, we aimed to establish a population distribution that evolved over time based on rewards, with a focus on initially training simpler tasks

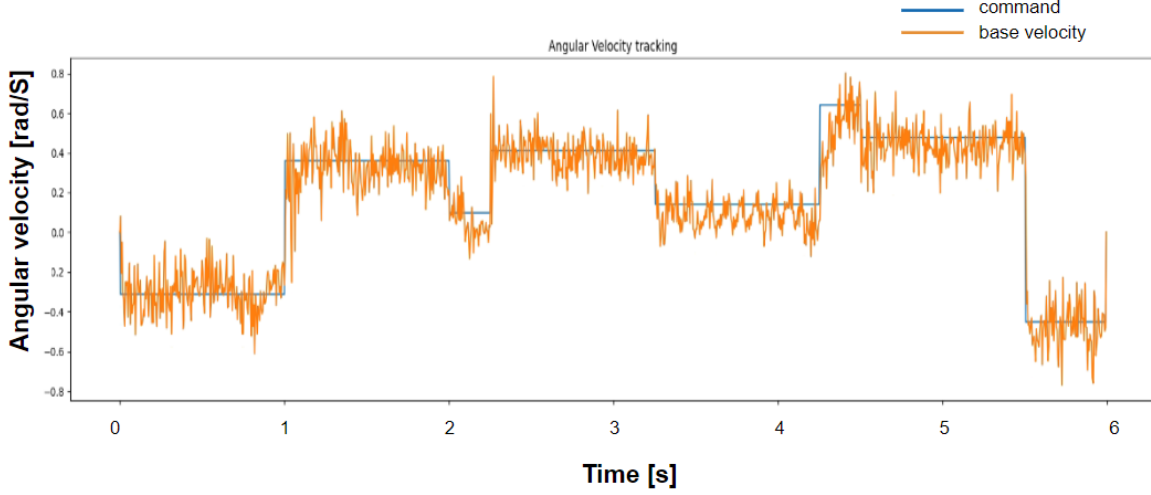


Figure 8: Angular velocity tracking plot

before progressing to more challenging ones.

We hypothesized that reaching a goal given its (x, y) coordinates would be the most difficult task. Therefore, during the early stages of training, we expected a higher number of agents to train on linear and angular velocity tracking tasks. Figure 9 illustrates the probabilities of sampling each task during the training iterations (iterations 0 to 800). At first all the probabilities are around $1/3$ (random probability) but around iteration 300 the linear velocity task probability starts increasing and is followed 150 iterations later by the angular velocity task, whereas the goal task decreases continually starting from iteration 300. This confirms our hypothesis, moreover we can see that linear velocity task is slightly easier than angular velocity task.

We also hypothesized that the goal task would have a better learning progress, which means a higher increasing rewards, in later stages of the training. Figure 10 illustrates the probabilities of sampling each task during the training iterations (iterations 3300 to 4300). We can see that from around iteration 3900 the goal task probability starts increasing while the other two probabilities decrease. Which means that the reward in the goal task is increasing while the other two are stagnating since they might have reached a maximum. This confirms our second hypothesis about the late training of the goal task.

5.4 Comparison between the two methods

In this section, we compare two methods we utilized to achieve the task of reaching a specific point in the xy plane. We conducted various setups and selected one that exhibited significant differences in complexity. Figure 11 depicts the goal setup, where the destination point at time zero is $(3, 2)$ in the robot's frame. The goal is positioned in front of the robot, resulting in a substantial rotation around the z -axis, exceeding 90° .

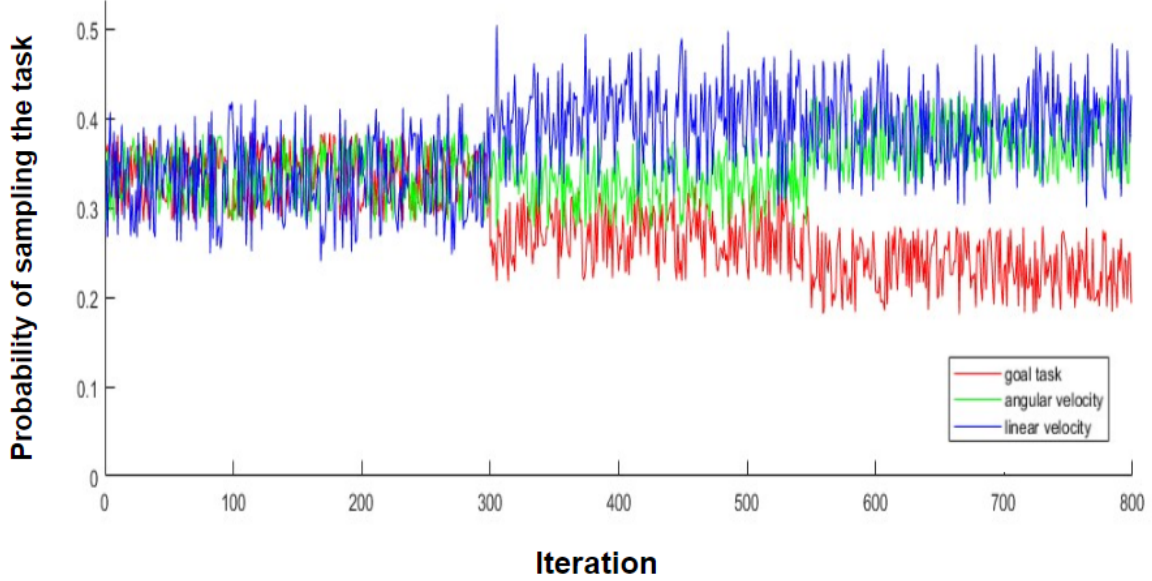


Figure 9: Task probability distribution (Part 1). The blue line represents the probability of training on the linear velocity task, the red line represents the probability of training on the goal task, and the green line represents the probability of training on the angular velocity task.

In the multi-task framework, the robot does not directly turn towards the goal; instead, it follows a circular path to reach the goal. This behavior is suboptimal as it entails an undesired contour. Additionally, the robot’s speed is relatively low (0.15 m/s), and its movement is unrealistic due to the hind feet sliding to prevent falling. It takes approximately 20 seconds for the robot to reach the goal point using this approach.

On the other hand, in the dual policy setup, we employ a simple controller. The robot initially rotates around the z-axis until it faces the goal point. Once aligned, it triggers linear translation, causing the robot to move backwards towards the goal. The robot stops when it reaches the desired point at (3, 2). This approach achieves a higher movement speed of around 0.4 m/s and follows the optimal path to reach the goal in approximately 10 seconds. Both videos demonstrating these methods can be found in Appendix section 8.

In conclusion, the dual policy method for locomotion in the xy plane yielded superior results compared to the multi-task method. However, it is important to note that the multi-task approach can still be improved by fine-tuning the reward functions. This process, though, can be time-consuming.

5.5 Payload

Figure 12 illustrates the payload capacity of the robot using the setup described in section 4.6. A comparison is made between the torque applied to the knees of the

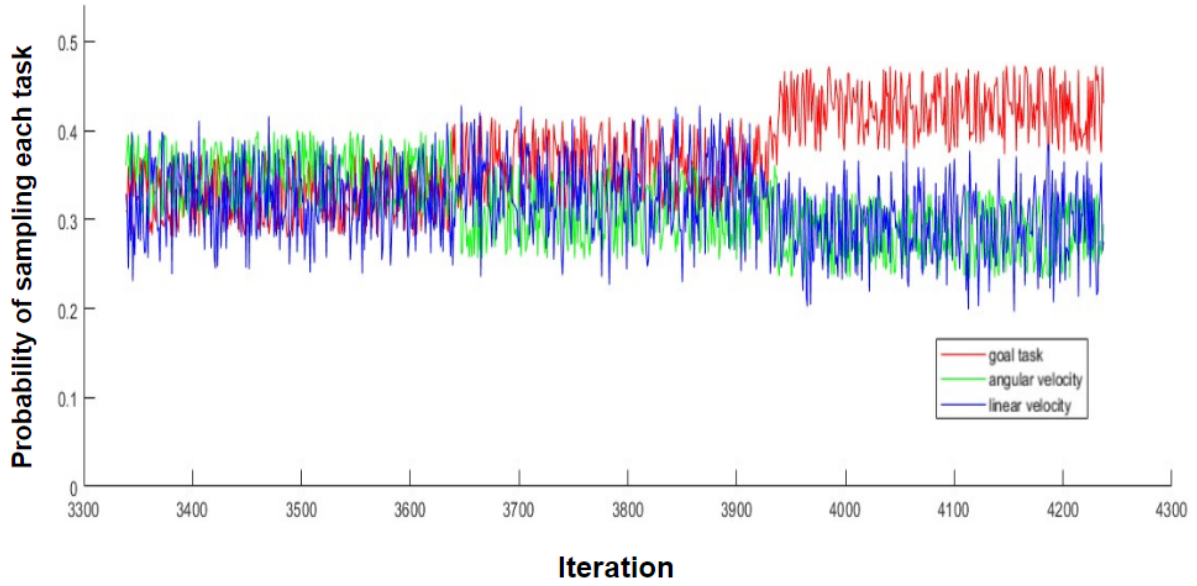


Figure 10: Task probability distribution (Part 2). The blue line represents the probability of training on the linear velocity task, the red line represents the probability of training on the goal task, and the green line represents the probability of training on the angular velocity task.

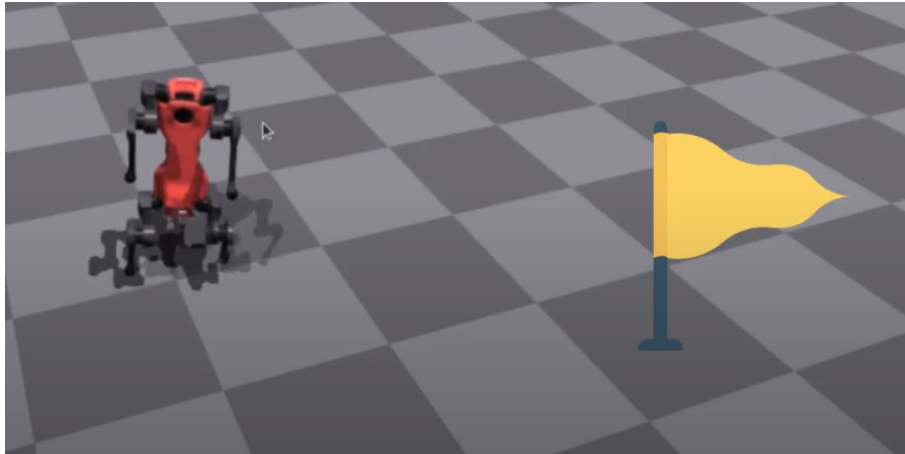


Figure 11: Goal comparison setup

ANYmal d robot in a bipedal configuration and the torque in our setup. The graph clearly shows that the torque exerted on the knees of the ANYmal d robot increases at a faster rate with respect to the weight in kilograms, compared to our robot in the described setup. In the ANYmal d robot's case, the torque continues to rise until it reaches the torque limit of the actuator, which is 90 Nm, at approximately 18 kg of weight. On the other hand, in our setup, the weight that the robot is capable of carrying increases up to 25 kg before reaching the torque limit.

Indeed, the difference in payload capacity between our robot in the given configuration and the ANYmal d robot arises from the utilization of the manipulator. Our robot employs its manipulator to distribute the weight across the three limbs in contact with the ground, as depicted in Figure 6. This distribution allows for a more balanced weight distribution and efficient torque utilization.

In contrast, the ANYmal d robot relies on distributing the weight between its two legs in a bipedal configuration, which may result in a relatively faster increase in torque and reach the torque limit at a lower weight threshold.

By incorporating the manipulator and distributing the weight among three contact points, our robot achieves a higher payload capacity and exhibits superior torque efficiency. This design enables our robot to carry heavier loads before reaching the torque limitation imposed by the actuators, showcasing its enhanced capability in handling weight-related tasks.

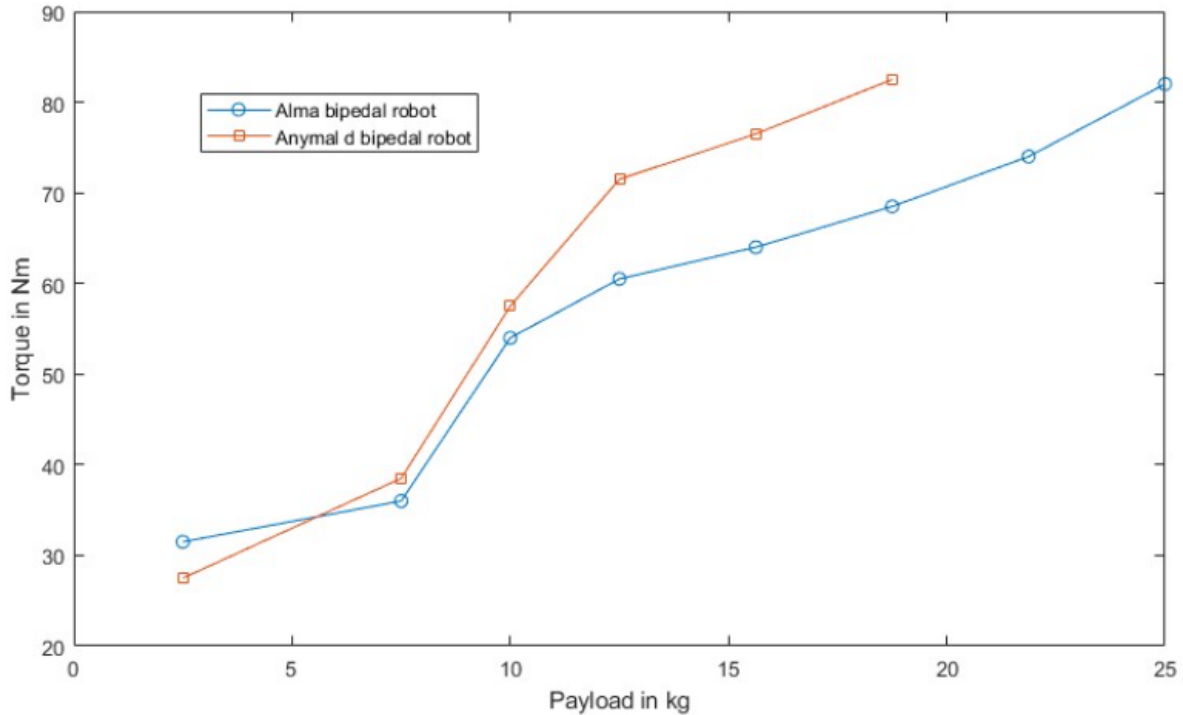


Figure 12: Mean torque applied to the knees in function of payload. The blue line represents the torque being applied to the knees in our setup where we use the manipulator as a support and the orange line represents the torque in the case of Anymal d robot in a bipedal setup (without manipulator)

6 Discussion

6.1 Upside Down Configuration Comparison

One topic of discussion could be comparing the advantages and disadvantages of two different orientations for performing the desired locomotion, as depicted in Figure 13, we call shoulder joint the joint circled in red. In the first orientation, referred to as the "normal" case, the manipulator is connected to the higher part of the base. In the second orientation, the manipulator is connected to the lower part of the base. One argument to consider is that in the "normal" case, the manipulator is closer to a singular configuration. However, it is important to thoroughly evaluate and compare the pros and cons of each configuration in order to make an informed decision.

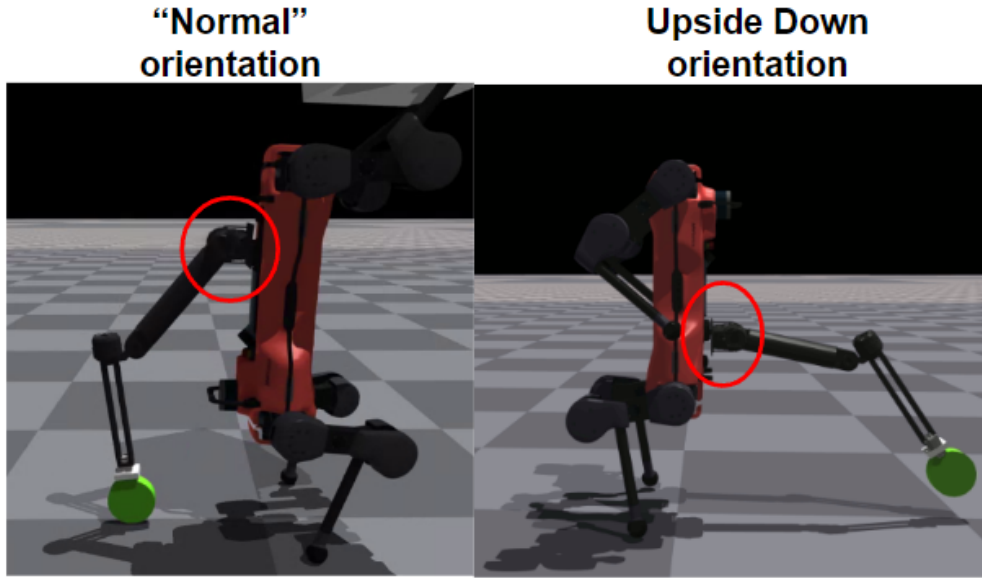


Figure 13: Two possible starting configurations, "shoulder joint" is circled in red

To be able to perform a fair comparison, we trained both environments to the same linear tracking velocity task used in 5.1.

The "normal" configuration is indeed closer to a singular configuration for the manipulator, the mean torque applied on the shoulder joint is of 12 Nm and the maximum torque is 22 Nm. The operation size in the x direction (base frame) is of 85 cm.

In the upside down configuration, the manipulator is further from the singular configuration, however the mean torque applied on the shoulder joint is of 21 Nm and the maximum torque is 27 Nm which is the maximum torque the actuator can deliver. The operation size in the x direction (base frame) is of 165 cm.

In conclusion, even though in the normal case the manipulator is closer to a singular configuration, after considering other aspects, we chose to continue using this orientation as our starting configuration.

6.2 Torque in knee during translation

As discussed in the problem formulation (Section 2), one of the factors to consider in our bipedal configuration is the torque applied in the legs' joints. We hypothesized that the actuators in the hip and knee joints would need to exert higher torque compared to a quadrupedal configuration due to the use of only two legs. Additionally, our solution requires the agent to frequently jump during translation, which can further increase the torque requirements.

In Figure 14, we observe the torque applied in the left knee joint for both the quadrupedal ANYmal configuration and our bipedal Alma robot configuration over time. The graph illustrates that in the case of quadrupedal locomotion, the torque remains below 40 Nm, while in our bipedal configuration, it frequently reaches the torque limit.

Furthermore, we notice that the frequency of torque spikes in our bipedal configuration is approximately 10 Hz, which is similar to our simulation refresh rate of 50 Hz. This implies that the torque spikes occur frequently and may pose challenges for accurate simulation, especially around torque limits.

Hence, this limitation in our pipeline indicates that the torque demands in our bipedal configuration are often close to or at the limit, and the simulation model may not accurately capture the dynamics around torque limits. It is crucial to address this limitation to ensure the stability and safety of the bipedal locomotion system when implementing it on the real robot.

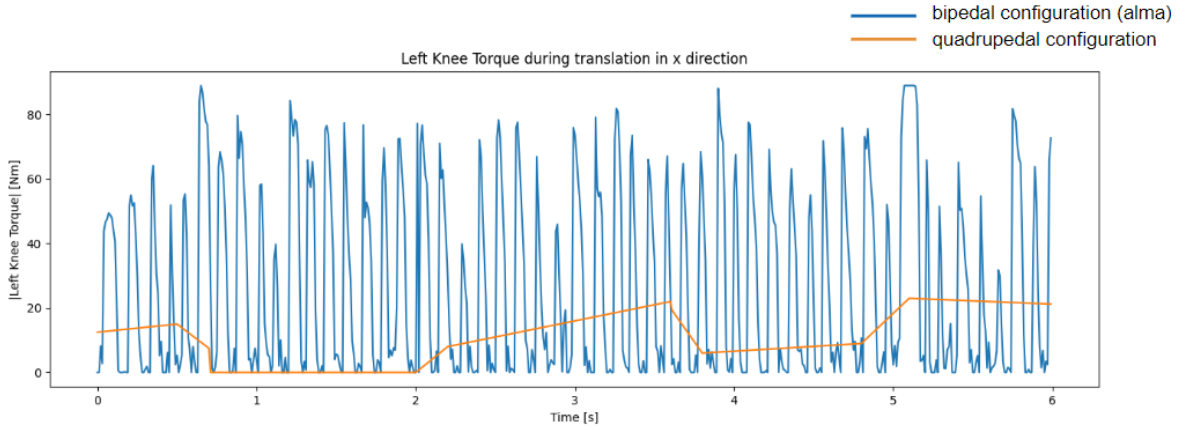


Figure 14: Left knee torque. In blue, our Alma robot in a bipedal configuration and in orange ANYmal robot in a quadrupedal setup

7 Conclusion

In conclusion, this semester project focused on exploring various approaches to improve the bipedal locomotion of the Alma robot, aiming to enable it to perform more

complex manipulation tasks. The implementation of a multi-task framework, where multiple tasks were learned using a single policy, did not yield satisfactory results initially. However, it is important to note that further improvements could be achieved by devoting more time to fine-tuning the reward functions. On the other hand, combining the linear and angular velocity tracking policies to accomplish a goal in the xy-plane reaching task exhibited promising and satisfactory outcomes. This approach showed potential in enhancing the robot’s locomotion capabilities. Nevertheless, it is important to acknowledge that our pipeline had a limitation in terms of the high torque observed during translation.

Another promising aspect is that we were able to achieve the desired locomotion using only 9 actuators in total, whereas other bipedal robots like the Atlas robot use 6 actuators on each leg, totaling 12 actuators.

Furthermore, we demonstrated that adding a manipulator had a positive impact not only on the robot’s stability during locomotion but also on its maximum weight-carrying capacity, thanks to the manipulator providing additional support.

Aspects to be worked on before the implementation on the real robot are to first improve the way the torque is distributed during locomotion, exploring other ways to perform translation without jumping while keeping the movement realistic, for example. We didn’t work on the hardware part but an advantage of our setup is that it is very simple to implement. In simulation, one aspect we couldn’t explore due to time limitations is the possibility to move on rough terrains (stairs for example) using our bipedal configuration.

Overall, the project provided valuable insights into different methods for enhancing bipedal locomotion and demonstrated both the challenges and potential solutions in achieving more intricate manipulation tasks with the Alma robot and its manipulator.

8 Appendix

Videos of the different experiments can be accessed through the following list :

- Kangaroo Tail Movement : <https://www.youtube.com/watch?v=LQJn3sIlYnI>
- Linear Velocity Tracking: [://drive.google.com/file/d/1bIIWS_wGmsQUUnFIu5DeubK2D0iKDmMg/view?usp=sharing](https://drive.google.com/file/d/1bIIWS_wGmsQUUnFIu5DeubK2D0iKDmMg/view?usp=sharing)
- Angular Velocity Tracking : https://drive.google.com/file/d/1vVctATsAsQNo9GKdM9feyHwfrl_I_gd3/view?usp=sharing
- Progressive Learning : <https://drive.google.com/file/d/1EqcPE1w4d-CgCsMRLE3VwVi1fI80E5st/view?usp=sharing>
- Dual Policy : <https://drive.google.com/file/d/1JeNCs7MOXyBJL6HjFkUrCkdEJaCItwsP/view?usp=sharing>

Link to the bitbucket branch : https://bitbucket.org/leggedrobotics/legged_gym/branch/feature/alma_bipedal_locomotion

Hyperparameter	Value
Initial Noise STD.	0
Actor Network	MLP with 3 hidden layers of [512, 256, 128] units
Critic Network	MLP with 3 hidden layers of [512, 256, 128] units
Value Loss Coefficient	1.0
Value Loss Clip	0.2
Entropy Coefficient	0.01
Number of learning Epochs per Iter.	1
Number of learning of mini Batches	1
Learning Rate	0.001
Schedule	fixed
Discount Factor γ	0.998
Desired KL div.	0.01
Max Grad. Norm	1.0
Number of Steps per Iter.	48
Number of Robot Envs	4096

Figure 15: Table presenting the hyperparameters used in our PPO algorithm

References

- [1] Jemin Hwangbo, Jonhoo Lee, AlexeyY Dosovistkiy, Dario Bellicoso , Vassilios Tsounis, Vladlen Koltun AND Marci Hutter: Learning Agile and Dynamic Motor Skills for Legged Robots
- [2] Eric Vollenweider, Marko Bjelonic, Victor Klemm, Nikita Rudin, Joonho Lee and Marco Hutter : Advanced Skills through Multiple Adversarial Motion Priors in Reinforcement Learning
- [3] Yuntao Ma, Farbod Farshidian, Takahiro Miki, Joonho Lee, Marco Hutter: Combining Learning-based Locomotion Policy with Model-based Manipulation for Legged Mobile Manipulators
- [4] Sebastian Blaes Marin Vlastelica Pogancic Jia-Jie Zhu Georg Martius: Control What You Can Intrinsically Motivated Task-Planning Agent
- [5] Cédric Colas, Pierre-Yves Oudeyer, Olivier Sigaud, Pierre Fournier, and Mohamed Chetouani. CURIOS: intrinsically motivated modular multi-goal reinforcement learning
- [6] Kangaroo Tails , by Dannie Holzer , California Academy of Sciences
- [7] J. Peters and S. Schaal, “Reinforcement learning of motor skills with policy gradients,” Neural networks : the official journal of the International Neural Network Society CURIOS: intrinsically motivated modular multi-goal reinforcement learning
- [8] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning

- [9] Nikita Rudin, Hendrik Kolvenbach, Vassilios Tsounis, Marco Hutter Cat-like Jumping and Landing of Legged Robots in Low-gravity Using Deep Reinforcement Learning
- [10] P Fankhauser, M Bjelonic, CD Bellicoso, T Miki, M Hutter Robust rough-terrain locomotion with a quadrupedal robot
- [11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov Proximal Policy Optimization Algorithms
- [12] Adrien Baranes and Pierre-Yves Oudeyer. Active Learning of Inverse Models with Intrinsically Motivated Goal Exploration in Robots. *Robotics and Autonomous Systems*, 61(1):69–73, January 2013.
- [13] Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents.
- [14] X. B. Peng, G. Berseth, K. Yin, and M. van de Panne, “Deeploco: dynamic locomotion skills using hierarchical deep reinforcement learning,”
- [15] Baranes, A. and Oudeyer, P.-Y. Active learning of inverse models with intrinsically motivated goal exploration in robots. *Robotics and Autonomous Systems*
- [16] Kaplan, F. and Oudeyer, P.-Y. Maximizing learning progress: an internal reward system for development.